



Reporte tecnico del proyecto Softlutionic

Sistema web superficial de facturacion, clientes y vista previa de factura

Proyecto preparado para defensa universitaria

Fecha del reporte: 29 de abril de 2026

1. Resumen general

Softlutionic es una maqueta web administrativa orientada a un sistema de facturacion. El proyecto esta construido con HTML, CSS y JavaScript puro. No utiliza backend ni base de datos real; los datos se simulan en arreglos de JavaScript dentro del archivo `js/data.js` . Esto permite demostrar flujos importantes durante una defensa universitaria sin depender de servidores, autenticacion ni persistencia.

Durante el trabajo realizado se completaron principalmente tres areas:

- Dashboard principal con navegacion, metricas, graficas simuladas y ultimas facturas.
- Modulo de Facturas con formulario, detalle de servicios, calculo de totales y vista previa imprimible.
- Modulo de Clientes con CRUD simulado, busqueda, filtros, tarjetas resumen y vista rapida.

2. Estructura de archivos trabajados

Archivo	Funcion dentro del proyecto
<code>index.html</code>	Pagina principal o dashboard del sistema.
<code>pages/facturas.html</code>	Modulo para crear una factura ficticia y generar una vista previa en formato carta.
<code>pages/clientes.html</code>	Modulo CRUD simulado para administrar clientes.
<code>css/styles.css</code>	Estilos generales del layout: sidebar, topbar, dashboard, tarjetas, paneles y tablas base.
<code>css/modules.css</code>	Estilos reutilizables para modulos internos, vista previa de factura e impresion.
<code>css/forms.css</code>	Estilos para formularios, campos, filtros y formularios de linea.
<code>css/tables.css</code>	Estilos complementarios para acciones en tablas, botones de icono y celdas vacias.
<code>js/app.js</code>	Renderiza las ultimas facturas en el dashboard.
<code>js/data.js</code>	Base de datos simulada y funciones auxiliares del sistema.
<code>js/facturas.js</code>	Controla el formulario de facturas, calculos, detalle y vista previa imprimible.
<code>js/clientes.js</code>	Controla el CRUD simulado del modulo Clientes.

3. Dashboard principal: `index.html`

El archivo `index.html` funciona como pantalla inicial del sistema. Su estructura esta dividida en un contenedor principal llamado `app-shell` , una barra lateral de navegacion y un area principal de contenido.

3.1 Encabezado HTML

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <link rel="stylesheet" href="css/styles.css" />
</head>
```

Este bloque define la codificación del documento, adapta la página a dispositivos móviles y carga la hoja de estilos principal. También se carga Font Awesome para mostrar iconos en botones, menus y tarjetas.

3.2 Sidebar o menu lateral

```
<aside class="sidebar">
  <a class="brand" href="index.html">
    
    <span>Softlutionic</span>
  </a>
  <nav class="menu">...</nav>
</aside>
```

Este bloque crea la navegación principal. Incluye el logo, el nombre del sistema y enlaces a Dashboard, Facturas, Clientes, Productos, Pagos, Reportes, Usuarios y Configuración. La clase `active` marca la página actual visualmente.

3.3 Topbar

El bloque `topbar` muestra el título de la pantalla, accesos rápidos, botones de mensajes, notificaciones y perfil. Su objetivo es dar apariencia de sistema administrativo real.

3.4 Tarjetas de metricas

```
<section class="metrics-grid">
  <article class="metric-card purple">...</article>
  <article class="metric-card orange">...</article>
</section>
```

Muestra indicadores como facturas emitidas, ingresos facturados, clientes activos y cuentas por cobrar. Son datos ficticios, pero ayudan a explicar el propósito del sistema.

3.5 Graficas simuladas

La gráfica de barras se construye con elementos HTML y variables CSS como `--bar-height`. La gráfica circular usa `conic-gradient`. No hay una librería externa de gráficas; todo es visual y estático.

3.6 Tabla de ultimas facturas

El cuerpo de la tabla usa el identificador `invoiceTable` . Este contenedor se llena desde `js/app.js` mediante JavaScript.

4. Modulo Facturas: `pages/facturas.html`

El modulo de facturas permite simular la creacion de una factura. Incluye formulario de datos generales, seleccion de cliente, contrato, detalle de servicios, calculo de totales y una vista previa imprimible en tamano carta.

4.1 Carga de estilos y scripts

```
<link rel="stylesheet" href="../../css/styles.css" />
<link rel="stylesheet" href="../../css/modules.css" />
<link rel="stylesheet" href="../../css/forms.css" />
<link rel="stylesheet" href="../../css/tables.css" />
...
<script src="../../js/data.js"></script>
<script src="../../js/facturas.js"></script>
```

Como la pagina esta dentro de la carpeta `pages`, las rutas usan `../` para regresar a la raiz del proyecto. Primero se carga la base de datos simulada y luego el controlador del modulo Facturas.

4.2 Formulario de datos de factura

El formulario contiene los campos principales: cliente, contrato, fecha de emision, fecha de vencimiento, tipo de factura, estado y observacion. Estos campos permiten que la vista previa tome datos actuales y se actualice visualmente.

4.3 Resumen de totales

El panel lateral muestra subtotal, IVA, descuento y total. Estos valores se actualizan desde JavaScript cuando se agregan servicios al detalle de la factura.

4.4 Detalle de servicios

```
<tbody id="invoiceDetailTable"></tbody>
```

Este cuerpo de tabla es manipulado por `facturas.js`. Cada servicio agregado genera una fila con codigo, nombre, cantidad, precio, IVA, total y una accion para eliminar.

4.5 Vista previa de factura impresa

```
<article class="letter-page" id="letterPreview">
  <header class="letter-header">...</header>
  <section class="letter-info-grid">...</section>
  <table class="letter-table">...</table>
</article>
```

Esta seccion representa una hoja tamaño carta. Contiene encabezado, logo, número de factura, cliente, datos del documento, detalle de servicios, observación y totales. El botón `Imprimir` llama a `window.print()`.

5. Logica de Facturas: `js/facturas.js`

5.1 Arreglo temporal de detalle

```
const invoiceItems = [];
```

Guarda en memoria los servicios agregados a la factura actual. No se guarda en base de datos; al recargar la página se pierde la información agregada.

5.2 Objeto de referencias DOM

```
const elements = {  
  invoiceCode: document.querySelector("#invoiceCode"),  
  customerSelect: document.querySelector("#customerSelect"),  
  invoiceDetailTable: document.querySelector("#invoiceDetailTable")  
};
```

Centraliza las referencias a elementos HTML que JavaScript necesita manipular. Esto evita repetir múltiples consultas al DOM en diferentes funciones.

5.3 Creación de celdas y estados

Las funciones `createCell` y `createStatusCell` construyen celdas de tabla de manera dinámica. `createStatusCell` asigna clases como `paid`, `pending` u `overdue` para colorear visualmente el estado.

5.4 Llenado de selects

`fillSelect` recibe un select, una lista de datos y una función para formar el texto de cada opción. Se usa para cargar clientes, contratos y servicios desde `data.js`.

5.5 Cálculo de totales

`updateTotals` recorre `invoiceItems` y suma subtotal, IVA, descuento y total. Luego escribe esos resultados en las tarjetas y en la vista previa.

5.6 Agregar servicios

`addService` obtiene el servicio seleccionado, cantidad y descuento. Luego usa `calcularTotalLinea`, función de `data.js`, para calcular subtotal, IVA y total de la línea.

5.7 Renderizado de vista previa

`renderPrintPreview` toma los datos actuales del formulario y los coloca dentro de la hoja carta. Esto permite que la factura impresa refleje cliente, fechas, contrato, observacion, servicios y totales.

6. Modulo Clientes: `pages/clientes.html`

El modulo Clientes fue creado para resolver la pagina que estaba vacia. Ahora contiene un CRUD simulado completo para administrar clientes dentro del navegador.

6.1 Estructura general

La pagina conserva el mismo layout que el dashboard y facturas: sidebar, topbar, `module-page`, tarjetas resumen, formulario principal, vista rapida y tabla. Esto mantiene consistencia visual entre modulos.

6.2 Tarjetas resumen

```
<strong id="totalClientsCard">0</strong>
<strong id="activeClientsCard">0</strong>
<strong id="newClientsCard">0</strong>
```

Estas tarjetas muestran total de clientes, clientes activos y clientes nuevos del mes. Los valores se calculan desde `clientes.js`.

6.3 Formulario de clientes

El formulario contiene campos para nombres, apellidos, telefono, correo, departamento, tipo de cliente, tipo de documento, numero de documento, fecha, estado y direccion. El campo oculto `clientIdInput` permite distinguir entre crear y editar.

6.4 Vista rapida

La vista rapida muestra una pequena ficha del cliente que se esta registrando o editando. Se actualiza mientras el usuario escribe.

6.5 Tabla y filtros

La tabla `clientsTable` muestra los registros. El buscador filtra por nombre, correo, telefono, direccion, departamento, tipo de cliente y documento. El segundo filtro permite ver todos, solo activos o solo inactivos.

7. Logica de Clientes: `js/clientes.js`

7.1 Objeto `clientElements`

Agrupar referencias al formulario, campos, botones, tabla, filtros y tarjetas. Esto permite que las funciones trabajen con elementos HTML ya identificados.

7.2 Normalizacion de datos

```
function ensureClientExtras() {
  clientes.forEach((cliente) => {
    if (!cliente.TipoCliente) {
      cliente.TipoCliente = cliente.Apellidos ? "Natural" : "Juridico";
    }
  });
}
```

Agrega propiedades visuales que no estaban en todos los clientes originales, como tipo de cliente, tipo de documento y numero de documento. Esto permite que la tabla tenga columnas mas completas.

7.3 Crear cliente

Al enviar el formulario, `saveClient` llama a `getClientFromForm`. Si el ID no existe, agrega el nuevo objeto al arreglo `clientes`. Si el ID existe, actualiza el registro correspondiente.

7.4 Editar cliente

`editClient` busca el cliente por ID, llena el formulario con sus datos y cambia el titulo a modo edicion. Al guardar, se reemplaza el objeto original en el arreglo.

7.5 Eliminar cliente

`deleteClient` elimina el cliente del arreglo usando `splice`. Es una eliminacion temporal en memoria; no afecta una base de datos real.

7.6 Busqueda y filtros

`getFilteredClients` transforma varios campos del cliente en una cadena de texto y compara contra lo escrito por el usuario. Tambien valida el filtro de estado.

7.7 Renderizado de tabla

`renderClients` limpia y reconstruye el cuerpo de la tabla con los clientes filtrados. Tambien agrega botones de editar y eliminar para cada fila.

8. Estilos generales: `css/styles.css`

Esta hoja contiene la base visual del sistema. Define variables CSS, layout principal, sidebar, topbar, tarjetas, paneles, graficas, tablas y estados.

8.1 Variables CSS

```
:root {  
  --bg: #f5f7fb;  
  --surface: #ffffff;  
  --sidebar: #121827;  
  --purple: #6d5dfc;  
}
```

Las variables permiten reutilizar colores y mantener coherencia visual. Si se cambia una variable, todas las secciones que la usan se actualizan.

8.2 Layout principal

`.app-shell` crea una distribucion de dos columnas: sidebar y contenido. En pantallas pequenas, las media queries transforman el layout para que sea usable.

8.3 Sidebar y navegacion

Las clases `.sidebar`, `.brand`, `.menu` y `.menu-link` controlan la barra lateral, logo, enlaces e iconos del menu.

8.4 Tarjetas y paneles

`.metric-card`, `.panel`, `.action-card` y `.table-panel` definen los contenedores principales. Incluyen bordes, fondo blanco, sombra y separacion.

8.5 Estados visuales

Las clases `.status.paid`, `.status.pending`, `.status.overdue` y `.status.neutral` dan color a estados como pagada, pendiente, vencida o inactiva.

9. Estilos de modulos: `css/modules.css`

Esta hoja fue usada para estilos compartidos entre Facturas y Clientes. Tambien contiene la configuracion de la hoja imprimible de la factura.

9.1 Clases reutilizables de modulos

`.module-page` organiza los bloques verticalmente. `.module-header` alinea titulo y acciones. `.invoice-layout` crea una columna principal y un panel lateral. `.summary-list` muestra pares de etiqueta y valor.

9.2 Vista previa carta

`.letter-page` simula una hoja tamaño carta. `.letter-header`, `.letter-brand`, `.letter-info-grid`, `.letter-table`, `.letter-bottom` y `.letter-footer` definen la estructura visual de la factura impresa.

9.3 Logo de la factura

El logo se configura en `.letter-brand img`. Se ajusto el tamaño, alineación a la izquierda y uso de `object-fit: contain` para mantener la proporción del archivo `assets/logo.png`.

9.4 Configuración de impresión

```
@media print {
  @page {
    size: letter;
    margin: 0;
  }

  body * {
    visibility: hidden;
  }

  #letterPreview,
  #letterPreview * {
    visibility: visible;
  }
}
```

Este bloque oculta todo el sistema al imprimir y deja visible solamente la factura. Así se evita que Chrome imprima capturas del sistema, sidebar, formularios o múltiples páginas.

10. Formularios y tablas: `css/forms.css` y `css/tables.css`

10.1 Formularios

`forms.css` define `.form-grid` para formularios de dos columnas, `.full-field` para campos que ocupan todo el ancho y `.line-form` para formularios horizontales como agregar servicios o filtrar clientes.

10.2 Tablas

`tables.css` define `.table-actions` para agrupar botones de acción, `.icon-button` para botones pequeños con iconos y `.muted-cell` para mensajes cuando no hay datos.

11. Datos simulados: `js/data.js`

Este archivo funciona como una base de datos falsa. Contiene arreglos como `departamentos`, `clientes`, `usuarios`, `servicios`, `contratos`, `facturas`, `detalleFactura` y `pagos`.

11.1 Funciones auxiliares

Incluye funciones como `formatoMoneda`, `obtenerClientePorId`, `obtenerNombreCliente`, `obtenerContratosPorCliente`, `calcularTotalLinea`, `calcularTotalesFactura` y `generarCodigoFactura`. Estas funciones evitan repetir logica en los modulos.

12. Limitaciones del proyecto

- No existe conexion real a una base de datos.
- Los registros creados, editados o eliminados se pierden al recargar la pagina.
- Los botones de guardar borrador o emitir factura son visuales, no realizan persistencia.
- La autenticacion y permisos de usuario son simulados.
- El objetivo principal es demostrar interfaz, flujo y estructura para una defensa universitaria.

13. Conclusiones

El proyecto Softlutionic quedo como una maqueta funcional de un sistema administrativo de facturacion. Las partes trabajadas permiten demostrar navegacion, dashboard, gestion de clientes, creacion de factura, calculos automaticos, detalle de servicios, vista previa en formato carta e impresion controlada. Aunque no posee backend, la estructura de archivos y la separacion entre HTML, CSS y JavaScript facilitan explicar el funcionamiento durante una presentacion academica.

Nota: el CRUD y la factura son simulados en frontend. Para convertirlo en un sistema real se deberia agregar backend, base de datos, validaciones de servidor y persistencia.